arXiv:2402.02619v10 [cs.LG] 3 Oct 2025

UNDERSTANDING ADDITION AND SUBTRACTION IN TRANSFORMERS

A PREPRINT

Philip Quirke
Martian
philip@withmartian.com

Clement Neo
NTU

Fazl Barez
University of Oxford, Martian
fazl@robots.ox.ac.uk

October 6, 2025

ABSTRACT

Transformers are widely deployed in large language models (LLMs), yet most models still fail on basic arithmetic tasks such as multidigit addition. In contrast, we show that small transformers trained from scratch can solve n-digit addition and subtraction with 99.999% accuracy. Building directly on prior work that uncovered addition circuits, we extend the analysis to subtraction and present a unified mechanistic account based on cascading carry and borrow circuits. Using a suite of 49 trained models, we apply systematic ablations and node-level constraints to validate the learned mechanisms, and release a reproducible interpretability toolkit for studying arithmetic circuits. Finally, surveying 180 publicly available LLMs, we find that only 7% can reliably perform addition, underscoring the gap between specialized small models and general-purpose LLMs. Our results show that arithmetic can be implemented exactly by tiny transformers, offering a tractable case study for mechanistic interpretability and a cautionary contrast with the persistent arithmetic failures of much larger models.

1 Introduction

Large language models (LLMs) achieve remarkable performance across diverse tasks, but still struggle with basic arithmetic. In a survey of 180 LLMs ranging from 1B to 405B parameters, 7% can reliably perform addition. The few that succeed typically rely on external tools, suggesting that the underlying models have not truly learned arithmetic (See App. E). In contrast, we show that transformers trained from scratch can master both addition and subtraction with near-perfect accuracy - without using external tools.

We construct (effectively infinite) synthetic training data enriched with the hardest edge cases, and train 49 small transformer models (2–3 layers, 3–4 attention heads) on 5 to 12-digit arithmetic. These models converge rapidly, reaching >99.999% accuracy. They succeed on the cascading carry one and borrow one cases where LLMs fail e.g. 555555555 + 444444448 = 1000000003. Our largest model has 10M parameters—over four orders of magnitude smaller than GPT-3—and trains to near-perfect accuracy in under an hour on a single GPU, providing an efficient, reproducible, and interpretable testbed for mechanistic analysis.

We introduce a novel, simple, mathematically exact algorithm for n-digit addition (Figure 1). The algorithm respects the transformer's left-to-right processing constraint, handling high-value digits before low-value ones. We also introduce a novel, exact algorithm for subtraction.

We then ask whether our trained models actually implement these algorithms. To do so, a model must realize each algorithmic subtask. We hypothesize the existence of specific calculation "nodes" capable of carrying out these subtasks, and develop code to automatically search for candidate nodes. We evaluate these candidates through systematic ablations. Each ablation is designed around a specific subtask, allowing us to precisely predict how the model's output should change when we intervene. The predictions match perfectly: intervening on the identified node produces exactly the expected effect. Every model we study contains nodes corresponding to all subtasks, though in some cases a single logical role is distributed across two attention heads.

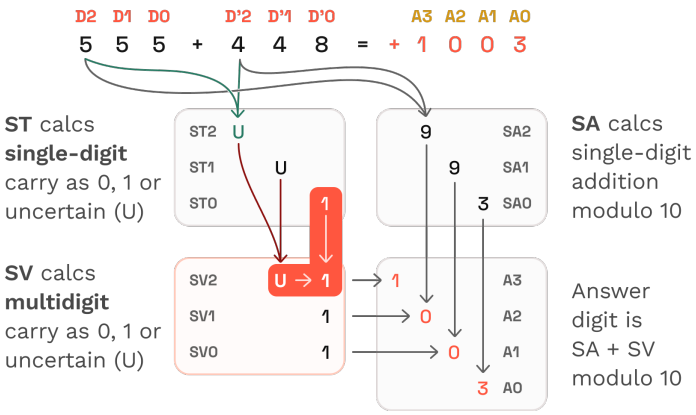

Figure 1: Our n-digit addition algorithm is mathematically sound. It uses 4 features: It calculates single-digit carry-one values (ST_n), combining them into multidigit carry-one values (SV_n). Any SV_n uncertain (U) values are refined to 0 or 1 over tokens (highlighted). By the "+" token, $SV2$ gives the A3 value as 0 or 1. The other answer digits A_n are calculated from SA_n and SV_{n-1} values.

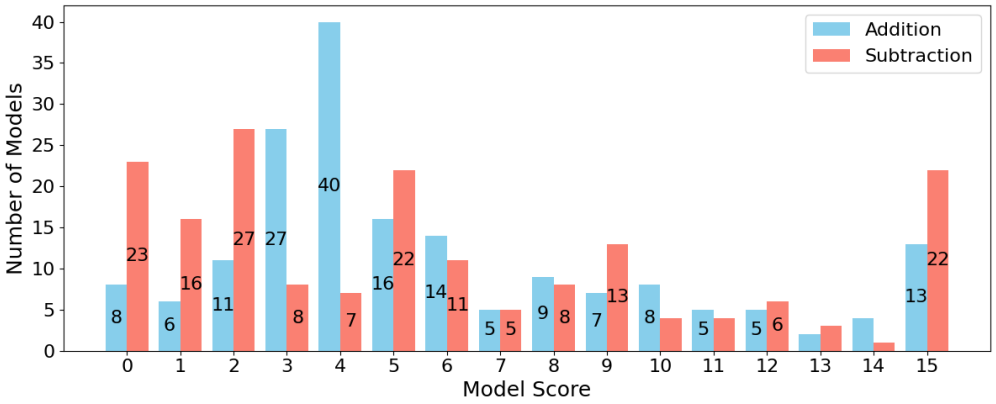

Figure 2: We score the addition and subtraction capability of 180 public LLMs. A score of 5 means the LLM handled two 5 digits numbers correctly but failed with 6 digit numbers. 7% of addition and 12% of subtraction models get the maximum test score 15. The top models can call external tools.

The algorithms also impose strict constraints on node ordering and relationships. For example, certain nodes must appear earlier in the computation sequence, while others depend on them downstream (see Figure 1). We test these constraints and find they are consistently satisfied across seeds, model sizes, and digit ranges. Together, these results show that all our accurate models converge on the same algorithmic solution.

To make this analysis reproducible, we release a general-purpose interpretability library. `https://github.com/PhilipQuirke/quanta_mech_interp`. It provides tools for (1) node characterization, search, and ablation, (2) systematic testing of node relationships and constraints, and (3) visualizations. It is a foundational tool used by this project `https://github.com/PhilipQuirke/quanta_maths`

Our contributions are fivefold: (i) a large-scale survey of LLM arithmetic capabilities, (ii) enriched datasets for addition and subtraction, (iii) 49 small transformer models trained to near-perfect accuracy, (iv) human-comprehensible exact addition and subtraction algorithms and (v) a reusable interpretability toolkit demonstrating that these models implement the algorithms.

We conclude that transformers can implement exact n-digit addition and subtraction through interpretable circuits. With appropriate training and data, larger LLMs could in principle adopt the same mechanisms.

2 Related Work

Mechanistic understanding of arithmetic in transformers. Early work by Nanda et al. [2023] revealed that one-layer models implement modular addition through discrete Fourier transforms, converting addition into rotations in frequency space - showing transformers can find unintuitive algorithms for basic mathematics. Quirke and Barez [2024] detailed an algorithm for n-digit addition in a single-layer model that achieved 99% accuracy, identifying a cascading carry-one failure mode. Subsequent work on training dynamics [Musat, 2024] and the finding that even random transformers exhibit algorithmic capabilities [Zhong and Andreas, 2024] suggest architectural biases toward systematic computation.

Architectural modifications and specialized models. Several approaches achieve near-perfect arithmetic accuracy through architectural changes. Reformatting inputs to present least-significant digits first by adding position embeddings and zero-padding strategies [McLeish et al., 2024]. Cho et al. [2024] align computational flow with carry propagation. Specialized models demonstrate superior performance: MathGLM's 10M parameter model achieves 100% accuracy on integer addition through step-by-step reasoning [Yang et al., 2023], while Qiu et al. [2024] reached 99.9% accuracy on 5-digit multiplication with a tiny transformer that outperforms GPT-4. Subtraction is much less studied, though Zhang et al. [2024] identified symmetries between addition and subtraction circuits, with the same attention heads handling both operations.

Production systems versus specialized models. The Claude 3.5 Haiku circuit analysis Lindsey et al. [2025] reveals how a frontier model implements addition. Rather than the clean n-digit algorithms found in specialized models, Claude employs a "bespoke" circuit combining lookup tables for memorized facts with magnitude estimation pathways. This distributed circuit achieves perfect accuracy on 2-digit addition using mechanisms distinct from both human methods and specialized models. The gap between Claude's complex implementation on short addition questions and the elegant n-digit algorithms in specialized models illustrates a fundamental challenge: production LLMs trained on diverse tasks develop hybrid strategies rather than the interpretable, generalizable algorithms that specialized models achieve.

3 Methodology

Transformer models learn addition algorithms different from traditional human methods. We define an alternative, mathematically-equivalent framework for addition and demonstrate our models implement this approach.

3.1 Mathematical Framework

For addition and subtraction of two n-digit numbers, we use the Fig. 3 notation. (Detail in App. A).

Figure 3: For 5-digit addition and subtraction, our notation for the main input tokens is $D4$, ..., $D0$ and $D'4$, ..., $D'0$. For output tokens it is $A6$, .., $A0$. For n-digits, we use the notation D_n, .., $D0$, D'_n, .., $D'0$ and A_n, .., $A0$.

3.2 Addition Algorithm Description

For addition, aligned with Quirke and Barez [2024], we define the "Base Add" subtask SA_n to compute the digit-wise sum modulo 10:

$$SA_n = (D_n + D'_n) \mod 10 \tag{1}$$

To solve n-digit addition with high accuracy, the framework must handle carry bits that cascade through multiple digits. For an autoregressive model to correctly predict the first answer digit in "555+448=+1003", as "1", it must cascade the carry bit from the *rightmost* digit to the *leftmost* digit in a single forward pass. This is particularly challenging for an autoregressive model that processes tokens from left to right. Aligned to this challenge we define two new subtasks.

First, we introduce subtask ST. It classifies a digit pair sum as *definitely* causing a carry (e.g. 6+7), definitely *not* causing a carry (e.g. 2+3), or *possibly* causing a carry (e.g. 5+4). The 5+4 case is uncertain as the digits sum to 9 and a carry from the next-lower-value digit would cause a carry: